

Quick Sort and Merge Sort

Quick Sort

- ▶ Quick Sort is a divide-and-conquer algorithm.
- ▶ - select a pivot element from the array.
- ▶ - partition the array into two subarrays based on the pivot.
- ▶ - recursively sort each subarray.
- ▶ Quick Sort has an average-case time complexity of $O(n \log n)$.
- ▶ However, its worst-case time complexity is $O(n^2)$.
- ▶ Quick Sort is an in-place sorting algorithm, meaning it doesn't require additional space beyond the input array.

Recursive Quicksort

```
void qsort(int v[], int left, int right) { // sort v[left...right]
    int i, last;
    void swap(int v[], int i, int j);

    if (left >= right) return; // there's nothing to sort

    // Partition v[left...right] using v[left] as the pivot

    last = left; // "v[last]" is the last element identified as
                // smaller than the pivot

    for (i = left + 1; i <= right; i++)
        if (v[i] < v[left]) // element smaller than the pivot
            swap(v, ++last, i); // is swapped with location last+1

    swap(v, left, last); // the pivot is moved to its correct place

    qsort(v, left, last - 1);
    qsort(v, last + 1, right);
}
```

Partition: Example

```
17 18 19 20 21 22 23 24 25 26 27  
23 24 31 27 22 18 32 14 37 35 36
```

```
left=17, right=27, last=17
```

```
i=18
```

```
i=19
```

```
i=20
```

```
i=21 23 22 31 27 24 18 32 14 37 35 36
```

```
last=18
```

```
i=22 23 22 18 27 24 31 32 14 37 35 36
```

```
last=19
```

```
i=23
```

```
i=24 23 22 18 14 24 31 32 27 37 35 36
```

```
last=20
```

```
i=25
```

```
i=26
```

```
i=27
```

```
14 22 18 23 24 31 32 27 37 35 36
```

```
after swap(v,17,20)
```

Merge Sort

- ▶ Merge Sort is also divide-and-conquer algorithm.
- ▶ It divides the input array into two halves.
- ▶ It then sorts each subarray recursively.
- ▶ Finally, it merges the sorted subarrays to produce the final sorted array.
- ▶ Merge Sort has a guaranteed worst-case time complexity of $O(n \log n)$.
- ▶ It requires additional space for merging, typically $O(n)$.

Recurrence Relation

- ▶ Let $T(n)$ be the time complexity of Merge Sort on an array of size n .
- ▶ Merge Sort divides the array into halves, so $T(n)$ can be expressed as:

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n)$$

- ▶ The $O(n)$ term represents the time to merge two sorted subarrays, which takes linear time.

Solving the Recurrence Relation

Given the recurrence relation:

$$T(n) = 2T\left(\frac{n}{2}\right) + cn$$

Let's unroll the recursion:

$$\begin{aligned}T(n) &= 2T\left(\frac{n}{2}\right) + cn = 2\left[2T\left(\frac{n}{4}\right) + c\left(\frac{n}{2}\right)\right] + cn \\&= 2^2T\left(\frac{n}{2^2}\right) + 2cn = 2^2\left[2T\left(\frac{n}{2^3}\right) + c\left(\frac{n}{2^2}\right)\right] + 2cn \\&= 2^3T\left(\frac{n}{2^3}\right) + 3cn = \dots = 2^kT\left(\frac{n}{2^k}\right) + kcn\end{aligned}$$

We continue until $\frac{n}{2^k} = 1$, which implies $k = \log_2 n$. Thus,

$$T(n) = 2^{\log_2 n} T(1) + \log_2 n cn = nT(1) + cn \log_2 n$$

where $T(1) = 0$ represents the base case. So, $T(n) = O(n \log n)$.

Merge

```
void merge(int arr[], int l, int m, int r) {  
    int n1 = m - l + 1;  
    int n2 = r - m;  
    int L[n1], R[n2];  
  
    for (int i = 0; i < n1; ++i) L[i] = arr[l + i];  
    for (int j = 0; j < n2; ++j) R[j] = arr[m + 1 + j];  
  
    int i = 0, j = 0, k = l;  
  
    while (i < n1 && j < n2)  
        if (L[i] <= R[j]) arr[k++] = L[i++];  
        else arr[k++] = R[j++];  
  
    while (i < n1) arr[k++] = L[i++];  
    while (j < n2) arr[k++] = R[j++];  
}
```

Merge Sort

```
void mergeSort(int arr[], int l, int r) {  
    if (l < r) {  
        int m = l + (r - l) / 2;  
  
        mergeSort(arr, l, m);  
        mergeSort(arr, m + 1, r);  
  
        merge(arr, l, m, r);  
    }  
}
```

Merge Sort

```
int main() {  
    int arr[] =  
        {38, 27, 43, 3, 9, 82, 10, 18,  
         75, 32, 6, 89, 47, 17, 57, 69};  
    int arr_size = sizeof(arr) / sizeof(arr[0]);  
  
    mergeSort(arr, 0, arr_size - 1);  
  
    // Display sorted array  
    for (int i = 0; i < arr_size; ++i)  
        printf("%d ", arr[i]);  
  
    return 0;  
}
```

What does this program print?

What does this program print?

```
#include <stdio.h>
char program[] = "#include <stdio.h>%cchar program[] = %c%s%c;%cint m
ain()%c{%cprintf(program, 10, 34, program, 34, 10, 10, 10, 10, 10,
10);%creturn 0;%c}%c";
int main()
{
printf(program, 10, 34, program, 34, 10, 10, 10, 10, 10, 10);
return 0;
}
```

A program that prints itself

```
$ gcc pself.c  
$ ./a.out > b.c  
$ diff pself.c b.c  
$
```

Pointers

Pointers and Addresses

- ▶ A pointer is a variable that contains the address of a variable.
- ▶ Use of pointers lead to more compact and efficient code.
- ▶ Pointers and arrays are closely related.
- ▶ Pointers + goto: recipe for complex code.
- ▶ Dangling references.
- ▶ With discipline, pointers can be used to achieve clarity and simplicity.

Pointers and Addresses

- ▶ A typical machine has consecutively addressed memory cells.
- ▶ A byte can be a `char`, two adjacent bytes can form a `short integer`, and four adjacent bytes can form a `long`.
- ▶ A pointer is a group of cells that can hold an address.

Pointers and Addresses

```
char *p;
```

It says that the expression `*p` is a char

```
char c;  
p = &c; /* p now points to c */
```

p:4A3BCC

345BD24

c:345BD24

'a'